

Lidar Wall Detection — Implementation Notes

This document describes the wall-detection and slope/orientation-following logic in `software/cpp/object_detection/object_detection.cpp`, and the important tunable variables if detection needs to be re-calibrated (new lidar unit, new arena lighting/ surfaces, different robot chassis, etc).

It covers two related but separate pipelines in the same file:

1. `Select_Wall()` / `Slope()` / `Distance_To_Wall()` — the robot's live decision-making path: "which wall is on my `front` / `right` / `left` / `behind`, what's its orientation, and how far away is it."
2. `postprocess_callback()` 's **lidar visualization** — a separate, independent wall detector used only to draw the debug "Lidar Coordinates" window. It exists purely to let a human see what the lidar sees; it does not feed the robot's control logic.

Both pipelines follow the same general recipe (filter noise → group points into candidate walls → score/pick the best one), but they are two independent implementations with their own tuning constants — a fix in one does not automatically apply to the other.

1. The `direction` side API

All wall queries take a single `direction` enum value instead of raw lidar angles:

```
enum direction{
    right = 1,
    left = -1,
    front = 2,
    behind = -2,
    invalid = 0
};
```

- `float Slope(const direction side)` — the selected wall's orientation, in degrees.
- `float Distance_To_Wall(const direction side)` — perpendicular distance (mm) from the robot to the selected wall's fitted line.

- `void Print_Slope(const direction side) / void Follow_Wall(const direction side)` — thin wrappers used by the robot's control loop.

Internally, `side` is mapped to a raw lidar angle (`middle_point`) in `Select_Wall()`:

```
switch (side) {  
    case right:  middle_point = 0;    break;  
    case left:   middle_point = 180;  break;  
    case front:  middle_point = 270;  break;  
    case behind: middle_point = 90;   break;  
}
```

This mapping is specific to the current lidar's physical mounting on the robot (currently the Oradar S2L). It was found empirically: the lidar's raw 0° reference does not line up with the robot's actual front, and it was determined by placing a single test object directly against the robot's front bumper and reading which raw angle lit up (a single isolated object gives an unambiguous reading; inferring the offset from a busy multi-wall room scene is much easier to get wrong).

If the lidar is replaced or reseated, re-derive this mapping — do not assume the old values still apply. Re-run the same test: an isolated object against a known side (front bumper is easiest), read its angle off the "Lidar Coordinates" window, and set that side's case to the observed angle.

2. `Select_Wall()` — the robot-facing wall detector

`Select_Wall(side)` is the shared core behind `Slope()` and `Distance_To_Wall()` — both call it and read different results off the same selection, so they can never disagree about which wall was chosen. Its steps, in order:

2.1 Build an angular window around the requested side

```
const int point = 70;           // total window width, in degrees  
const int half_point = point / 2; // window is centered on middle_point, not on
```

The window spans `[middle_point - half_point, middle_point + half_point)`. It must stay well under 90° because the four sides are only 90° apart — too wide a window and adjacent sides start reading each other's walls (this happened once when `point` was 170; narrowing it to 70 fixed cross-talk between sides at the cost of some coverage on very long walls near a direction boundary).

```
const int middle_exclusion_deg = 3;
```

Points within 3° of the exact center angle are dropped — that spot is often a seam or corner between two walls, and including it can pull in a small noisy cluster.

2.2 Filter out the robot's own chassis

```
const float min_range_mm = 150.0f;
```

Anything closer than this is almost certainly the robot's own body/mount, not an environmental wall. These near-origin returns are trivially "clean" (a rigid nearby surface fits a line almost perfectly), so without this filter they would out-score real walls every time. **If the lidar is mounted differently, or a new chassis has a different self-obstruction distance, re-check this value.**

2.3 Radius Outlier Removal (ROR)

```
const float ror_radius_mm = 300.0f;  
const int   ror_min_neighbors = 3;
```

A point is dropped if it has fewer than `ror_min_neighbors` other points within `ror_radius_mm` of it. This prevents a single stray noisy point from triggering a false gap/corner split and shattering a real wall into fragments.

```
const int edge_pad_deg = 15;
```

ROR needs neighbors on *both* sides of a point to judge it fairly. A real wall point sitting at the edge of the window has neighbors just *outside* the window — without padding, it looks falsely isolated and gets dropped, truncating real walls right at the window boundary. So the code collects a padded range purely to support neighbor counting, then trims back to the actually-requested window (`pt_is_core`) before scoring.

2.4 Segment into candidate walls

```
const float gap_threshold_mm = 60.0f;
```

Gap-based segmentation: a new wall segment starts wherever consecutive points jump apart by more than this distance.

```
const float corner_split_threshold_mm = 40.0f;
const int   corner_margin_points = 2;
```

Each gap-segment is further split at corners — an orientation change with *no* range gap (two walls meeting at a corner have continuous range but different angle) — using a recursive max-perpendicular-deviation split (Iterative End-Point Fit). A few points straddling the detected corner vertex are dropped, since corner-adjacent returns tend to be noisy/transitional.

2.5 Score and filter candidates

```
const float max_wall_residual_mm = 15.0f;
const int   min_wall_points = 6;
const float min_wall_extent_mm = 200.0f;
```

- **min_wall_points** : segments smaller than this are excluded entirely. This must be high enough that residual is actually meaningful — a 2-3 point segment is geometrically almost a perfect line no matter what, so tiny fragments are structurally biased toward looking "clean" even though they prove nothing.
- **max_wall_residual_mm** : a segment "qualifies" if its RMS perpendicular distance to its own best-fit line is under this. Qualifying segments always outrank non-qualifying ones.
- **min_wall_extent_mm** : a segment must span at least this far end-to-end to count as a wall. This exists because a compact object (a box, a pillar, another robot) can rack up enough points within the ROR radius to pass **min_wall_points** and still look "clean" by residual — neither of those checks can tell a wall from a blob on their own. Requiring physical extent directly excludes objects, independent of point count or residual.

Among qualifying candidates, ranking prefers **more points** first, then lower residual as a tiebreaker only — point count is never overridden by residual alone, since that's what previously let tiny clean fragments beat real, longer (slightly noisier) walls.

2.6 Per-side wall tracking (avoids flipping to the adjacent wall)

```
const float wall_track_max_match_mm = 300.0f;
const float wall_track_max_angle_diff_deg = 20.0f;
```

Once a wall has been chosen for a given **side**, the next call prefers to keep following *that same physical wall* (matched by centroid position + orientation) rather than always snapping to whichever segment scores best this frame. This matters because a few degrees of robot rotation can shift the fixed angular window enough that an adjacent wall briefly outscores the wall actually being

followed, which previously caused a visible flip mid-turn. If the previously-tracked wall can't be matched within `wall_track_max_match_mm` / `wall_track_max_angle_diff_deg` (e.g. it genuinely left the window), the code falls back to the best-scoring candidate and starts tracking that instead — so it still self-recovers.

Tracking state is keyed per- `side` (a `static std::unordered_map<int, WallTrackState>` inside `Select_Wall`), so `front` / `right` / `left` / `behind` each track their own wall independently.

2.7 Merge a split wall, but not a real corner

```
const float max_merge_angle_diff_deg = 12.0f;
```

The chosen wall's points are merged with one other candidate segment (the top-scoring one, or the second-best if the chosen wall already is the top scorer) **only if** the two segments' independently-fit orientations agree within this threshold. This handles a wall that got split into two segments by a sensor dropout without incorrectly merging two genuinely different walls meeting at a corner (which would fit a meaningless line through neither surface).

2.8 Outputs

- `Slope(side)` returns `Fit_Line_Orientation()` on the final chosen points — a total-least-squares (PCA) line-direction fit, robust to which axis has more variance.
- `Distance_To_Wall(side)` returns the perpendicular distance from the robot's origin to that same fitted line (not just the centroid's range, since the centroid can sit off to one side of the perpendicular foot).
- `Publish_Slope_Used_Angles()` records exactly which raw lidar angles fed the result, so the visualization can highlight precisely those points (see below).

3. The visualization's wall detector (`postprocess_callback`)

This is a **separate implementation** that scans the *entire* 360° buffer (not a narrow window around one side) purely to draw the "Lidar Coordinates" debug window. It is not used by `Slope()` / `Distance_To_Wall()` and tuning it does not affect robot behavior — only what you see on screen.

```
const float ror_radius_mm = 100.0f;  
const int   ror_min_neighbors = 10;
```

Its own ROR pass (different radius/neighbor values than `Select_Wall` 's).

```
const float bpd_lambda_deg = 10.0f;    // min incidence angle assumed for a flat
const float bpd_sigma_r_mm = 10.0f;    // range measurement noise estimate
```

Uses the Adaptive Breakpoint Detector (Borges & Aldon) instead of a flat gap threshold: the maximum allowed gap between consecutive points (`D_max`) is computed per-point from range and incidence-angle geometry, rather than being a single fixed distance. **A smaller `bpd_lambda_deg` makes `D_max` larger** (more tolerant of gaps, less likely to split) — this was raised from 5° to 10° after a nearby test object failed to split from an adjacent wall (at 5°, a ~450mm-range point had `D_max` ≈142mm, generous enough to merge a real object into the wall next to it).

```
const int max_walls = 8;
const float wall_track_max_match_mm = 400.0f;
```

Keeps only the `max_walls` largest segments as "real" walls (everything else is drawn as grey clutter). Also does its own centroid-based color tracking (`wall_tracks`) so the same physical wall keeps the same color across frames instead of flickering when two similarly-sized walls swap size-rank — conceptually similar to `Select_Wall` 's per-side tracking in §2.6, but keyed by color slot rather than by `side` , and only used for rendering.

Highlighted-wall distance overlay

Whenever `Slope()` / `Distance_To_Wall()` has highlighted a wall (the black-outlined points in the debug view — driven by `Publish_Slope_Used_Angles`), the visualization independently computes the perpendicular distance from the origin to that highlighted wall's fitted line, and:

- prints it to the terminal: `[Highlighted wall] distance=512mm`
- draws it as text on the canvas, in the same color as the highlighted wall.

This is a display-only echo of the same perpendicular-distance math used in `Distance_To_Wall()` — it's recomputed here only so the overlay works even when nothing has called `Distance_To_Wall()` this frame.

4. Mathematical formulas

This section gives the actual math behind each step in §2/§3, using the raw lidar return (`angle_deg` , `range_mm`) as the starting point. Points are indexed `i = 1..N` within whatever set

is being processed (a window, a segment, etc.).

4.1 Polar → Cartesian conversion

Every raw (angle, range) return is converted to a 2D point in the lidar's own frame (mm), robot origin at (0, 0), before any of the steps below run:

```
x_i = range_i * cos(angle_i)
y_i = range_i * sin(angle_i)
```

4.2 Radius Outlier Removal (ROR)

For each point i , count how many other points j fall within `ror_radius_mm`:

```
neighbor_count(i) = | { j != i : ||p_i - p_j|| <= ror_radius_mm } |

||p_i - p_j|| = sqrt( (x_i - x_j)^2 + (y_i - y_j)^2 )    (Euclidean distance)
```

Point i is kept only if `neighbor_count(i) >= ror_min_neighbors`; otherwise it's discarded as an outlier.

4.3 Gap-based segmentation

A new segment starts between consecutive points $i-1$ and i whenever:

```
||p_i - p_{i-1}|| > gap_threshold_mm
```

4.4 Adaptive Breakpoint Detector (visualization only, Borges & Aldon 2004)

Instead of one fixed gap threshold, the maximum allowed distance between consecutive points `D_max` is derived per-point from range and an assumed minimum surface incidence angle λ (`bpd_lambda_deg`):

```
 $\Delta\phi = \text{angle}_i - \text{angle}_{i-1}$                                 (angular step between returns)

D_max = r_{i-1} * sin( $\Delta\phi$ ) / sin( $\lambda - \Delta\phi$ ) + 3 *  $\sigma_r$     if  $\Delta\phi < \lambda$ 
D_max = -1 (never split; treat as "infinite gap allowed")    if  $\Delta\phi \geq \lambda$ 
```

where r_{i-1} is the previous point's range and σ_r is `bpd_sigma_r_mm` (assumed range measurement noise). A new segment starts if $||p_i - p_{i-1}|| > D_{max}$. Intuition: a flat surface seen at a shallow (near-parallel) incidence angle naturally produces bigger point-to-point spacing than one viewed head-on, at the same range — D_{max} grows accordingly so a real continuous wall isn't falsely split, while λ sets the floor below which any gap at all is treated as a break.

Why a smaller λ makes the detector more tolerant: for fixed $\Delta\phi < \lambda$, shrinking λ shrinks $(\lambda - \Delta\phi)$, which shrinks $\sin(\lambda - \Delta\phi)$ (for small angles, $\sin$ is roughly linear), which appears in D_{max} 's *denominator* — so D_{max} grows. A larger D_{max} means more real gaps get "explained away" as an artifact of viewing angle rather than a genuine break in the surface, which is exactly what let a nearby test object merge into the wall next to it at $\lambda = 5^\circ$.

4.5 Corner splitting (Iterative End-Point Fit / split-and-merge)

For a segment with endpoints p_1 and p_n , every interior point p_k is scored by its perpendicular distance to the straight line $p_1 \rightarrow p_n$:

```
line_dir = p_n - p_1
dist(p_k) = | line_dir × (p_k - p_1) | / ||line_dir||

where a × b = a.x * b.y - a.y * b.x      (2D cross product, gives a scalar)
```

The point with the largest $\text{dist}(p_k)$ is the candidate corner. If $\max(\text{dist}) > \text{corner_split_threshold_mm}$, the segment is split there (dropping `corner_margin_points` points on each side of the split index), and the same test is applied recursively to both halves.

4.6 Total Least Squares (PCA) line orientation fit

Why not ordinary least squares? Fitting $y = mx + b$ minimizes *vertical* distance and blows up ($m \rightarrow \infty$) for a wall running nearly parallel to the y-axis — which happens constantly here, since a wall can face the robot at any angle. Total Least Squares instead minimizes *perpendicular* distance to the line, which has no preferred axis and works identically at any orientation. This is exactly what PCA gives you: the line through the centroid along the direction of maximum variance is also the line that minimizes total squared perpendicular distance to it.

Step 1 — center the points. Given the mean $(\bar{x}, \bar{y})$ of the segment's N points, form the centered coordinates:

$$\begin{aligned} dx_i &= x_i - \bar{x} \\ dy_i &= y_i - \bar{y} \end{aligned}$$

Step 2 — build the 2×2 covariance (scatter) matrix.

$$C = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}$$

$$S_{xx} = \sum dx_i^2, \quad S_{yy} = \sum dy_i^2, \quad S_{xy} = \sum dx_i * dy_i$$

(these are the unnormalized covariances — dividing by `N` would give the usual covariance matrix, but the scale doesn't affect the eigenvectors, so the division is skipped).

Step 3 — find the eigenvector of `C` with the largest eigenvalue. For a general point cloud, PCA proceeds by eigen-decomposing `C`. For a symmetric 2×2 matrix this has a closed form instead of needing iterative eigen-solvers:

$$\text{eigenvalues: } \lambda_{\pm} = (S_{xx} + S_{yy})/2 \pm \sqrt{((S_{xx} - S_{yy})/2)^2 + S_{xy}^2}$$

`λ+` (the larger root) is the variance *along* the wall's direction; `λ-` is the (ideally small) variance *across* it — i.e. the noise. The eigenvector belonging to `λ+` points along the wall. Rather than solving $(C - \lambda_+ I) v = 0$ directly for that eigenvector, the code uses the standard double-angle identity for diagonalizing a symmetric 2×2 matrix by rotation: the rotation angle `θ` that diagonalizes `C` (equivalently, the direction of its dominant eigenvector) satisfies:

$$\tan(2\theta) = 2 * S_{xy} / (S_{xx} - S_{yy})$$

$$\theta = 0.5 * \text{atan2}(2 * S_{xy}, S_{xx} - S_{yy})$$

using `atan2` (rather than plain `atan`) so the correct quadrant is picked up automatically across the full 180° range of possible wall orientations.

`θ` (converted to degrees) is the line's orientation — the axis of maximum variance through the points, equivalently the total-least-squares best-fit line direction. Because a line and its 180°-rotated reverse describe the same orientation, `θ` is only meaningful modulo 180° (see the wraparound handling in §4.9).

4.7 Line-fit residual (RMS perpendicular error)

Using the same θ from §4.6, the unit normal to the fitted line is $n = (-\sin \theta, \cos \theta)$. Each point's signed perpendicular distance to the line through the centroid is:

$$r_i = dx_i * n.x + dy_i * n.y$$

$$\text{residual} = \sqrt{(1/N) * \sum r_i^2} \quad (\text{RMS of the } r_i)$$

A segment "qualifies" as a clean wall if `residual <= max_wall_residual_mm`.

4.8 Perpendicular distance from the robot to the wall

`Distance_To_Wall()` and the visualization's highlighted-wall overlay both compute the distance from the origin (the robot) to the wall's fitted *line* (not to the wall's centroid point), using the same unit direction vector $\text{dir} = (\cos \theta, \sin \theta)$ from §4.6 and the segment's centroid $(\bar{x}, \bar{y})$:

$$\text{distance} = | \bar{x} * \text{dir}.y - \bar{y} * \text{dir}.x |$$

This is the magnitude of the 2D cross product of the centroid position with the line's direction vector — geometrically, the length of the perpendicular dropped from the origin onto the infinite line through the centroid along `dir`.

4.9 Wall-identity matching (tracking)

Both `Select_Wall`'s per-side tracking (§2.6) and the visualization's color tracking match a candidate segment `c` against a previously-tracked wall `t` using two independent criteria that must both pass:

```
position_match: ||centroid_c - centroid_t|| <= wall_track_max_match_mm
```

```
angle_diff = | orientation_c - orientation_t |
```

```
if angle_diff > 90°: angle_diff = 180° - angle_diff      (line orientation is m  
                                                         a line and its reve  
                                                         the same orientatio
```

```
orientation_match: angle_diff <= wall_track_max_angle_diff_deg
```

Among all candidates satisfying both, the one with the smallest `position_match` distance is chosen as the continuation of the tracked wall.

4.10 Angle classification (side labeling)

Given a wall's centroid, its angle from the robot is `atan2( $\bar{y}$ ,  $\bar{x}$ )` (normalized to `[0°, 360°)`). To classify which `side` it's nearest to, the angular difference to each reference angle (`front=270°, right=0°, left=180°, behind=90°`) is computed with wraparound handled the same way as §4.9:

```
diff(a, ref) = | a - ref |
if diff > 180°: diff = 360° - diff

side = argmin_ref diff(a, ref)
```

5. Quick reference: constants likely to need re-tuning

Constant	Location	Current value	Purpose
<code>middle_point</code> per <code>side</code>	<code>Select_Wall</code> switch	front=270, right=0, left=180, behind=90	Lidar-mount-specific angle offset — re-derive if the lidar is replaced/reseated
<code>point</code>	<code>Select_Wall</code>	70°	Angular window width per side
<code>min_range_mm</code>	<code>Select_Wall</code>	150mm	Excludes robot's own chassis
<code>ror_radius_mm</code> / <code>ror_min_neighbors</code>	<code>Select_Wall</code>	300mm / 3	Outlier rejection
<code>gap_threshold_mm</code>	<code>Select_Wall</code>	60mm	Gap-based wall segmentation
<code>corner_split_threshold_mm</code>	<code>Select_Wall</code>	40mm	Corner detection sensitivity
<code>min_wall_points</code>	<code>Select_Wall</code>	6	Minimum points to be considered a wall
<code>max_wall_residual_mm</code>	<code>Select_Wall</code>	15mm	Max noise for a wall to "qualify"
<code>min_wall_extent_mm</code>	<code>Select_Wall</code>	200mm	Minimum physical size (excludes

Constant	Location	Current value	Purpose
			objects)
<code>wall_track_max_match_mm / _angle_diff_deg</code>	<code>Select_Wall</code>	300mm / 20°	Wall-identity tracking per side
<code>max_merge_angle_diff_deg</code>	<code>Select_Wall</code>	12°	When two segments merge into one wall
<code>ror_radius_mm / ror_min_neighbors</code>	<code>postprocess_callback</code> (visualization only)	100mm / 10	Visualization-only outlier rejection
<code>bpd_lambda_deg</code>	<code>postprocess_callback</code> (visualization only)	10°	Visualization-only gap tolerance
<code>max_walls</code>	<code>postprocess_callback</code> (visualization only)	8	How many walls get distinct colors on screen

When re-tuning, change one constant at a time and re-test against a known scene — several of these interact (e.g. widening `point` reduces directional selectivity; loosening `ror_radius_mm` can let a small object pass `min_wall_extent_mm`'s neighbor-density prerequisite).